



Antes de empezar...

- Comentarios tarea 1

Inteligencia Artificial

Clase 6: Metaheurísticas



Para el día de hoy

- Hillclimber
- Recocido simulado
- Búsqueda tabú
- Algoritmos evolutivos





Los métodos hasta ahora...

- Construyen las soluciones un paso a la vez
- Tienen garantías de optimalidad (en la mayoría de los casos)
 - Búsqueda en profundidad (*)
 - Búsqueda en anchura
 - Búsqueda iterativa
 - Búsqueda de costo uniforme
 - Búsqueda voraz (*)
 - A*
 - Búsqueda con backtracking (ramificación y poda)
 - Backtracking
 - Ordenamiento
 - Filtrado



Búsqueda local

- Mover las configuraciones haciendo movimientos locales
- Trabaja con asignaciones completas
- Generalmente más rápido y eficiente en memoria
- Incompleto y suboptimo
- Problemas de satisfacción
 - Iniciar con una configuración no factible
 - Moverse hacia una solución factible
- Optimización
 - Iniciar con soluciones subóptimas
 - Moverse hacia una solución optima

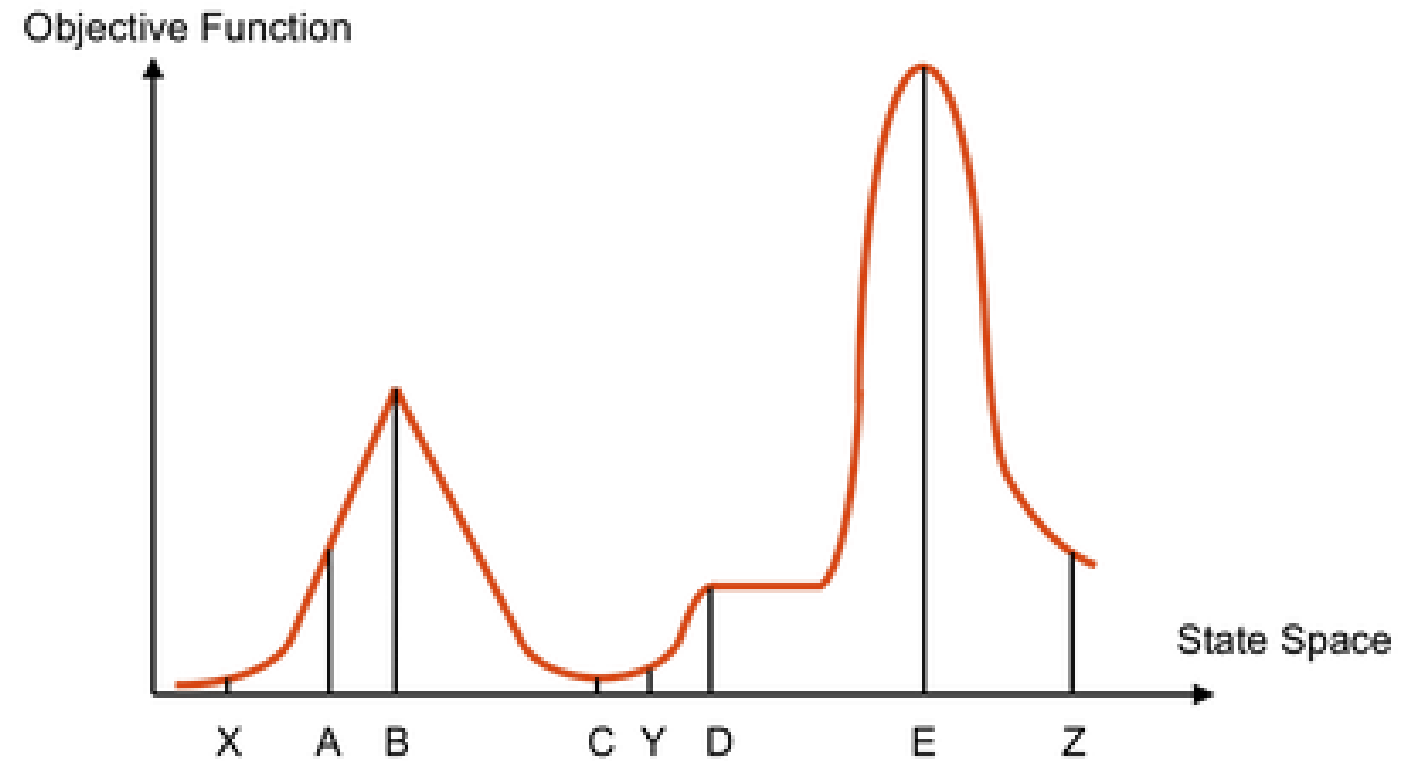
Hill climbing

- La idea
 - Iniciar en cualquier lugar
 - Moverse al mejor estado vecino
 - Si no hay un mejor vecino, terminar
- Las preguntas
 - ¿Es completo?
 - ¿Es óptimo?
- ¿Cuáles son sus ventajas?
- ¿Cuáles son sus desventajas?



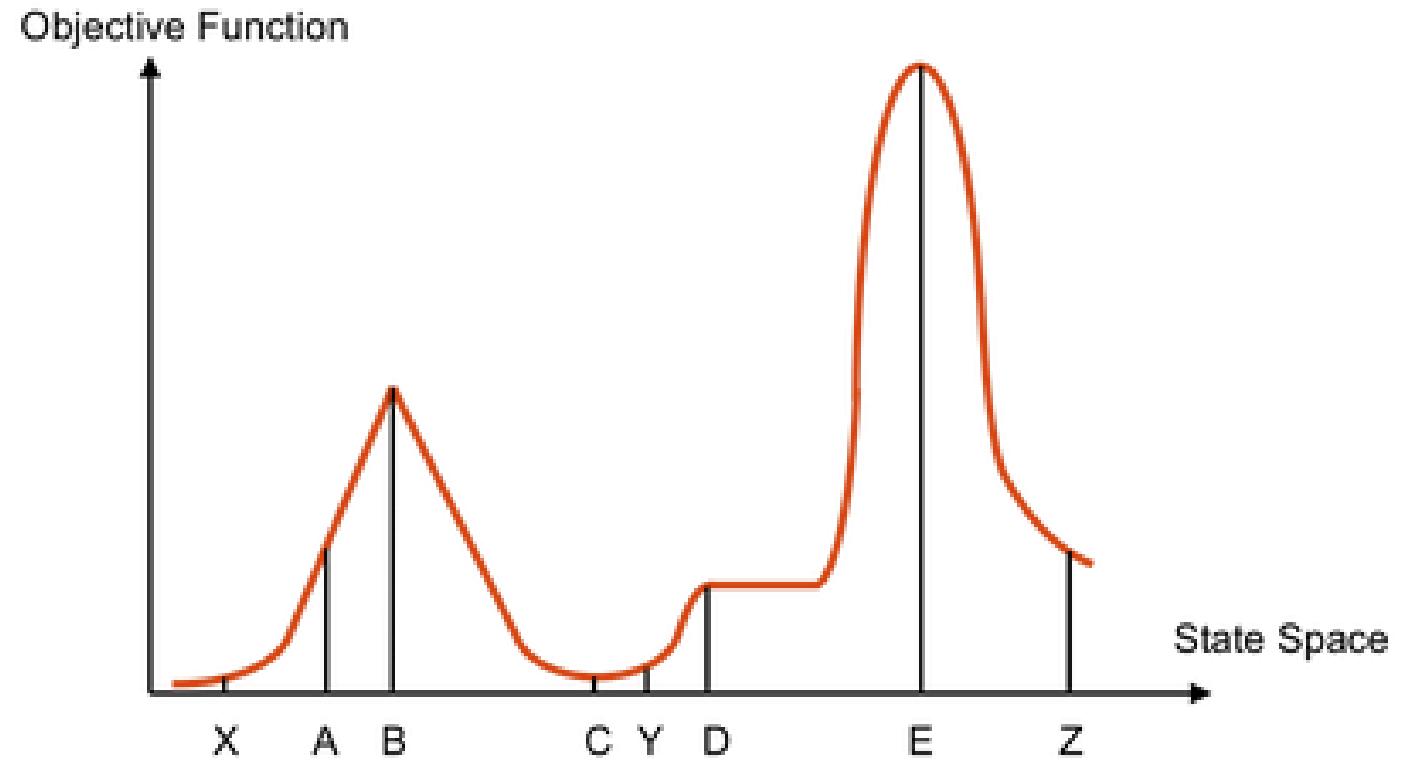
Un quizz

- En donde terminas iniciando en:
 - X
 - Y
 - Z



Búsqueda tabú

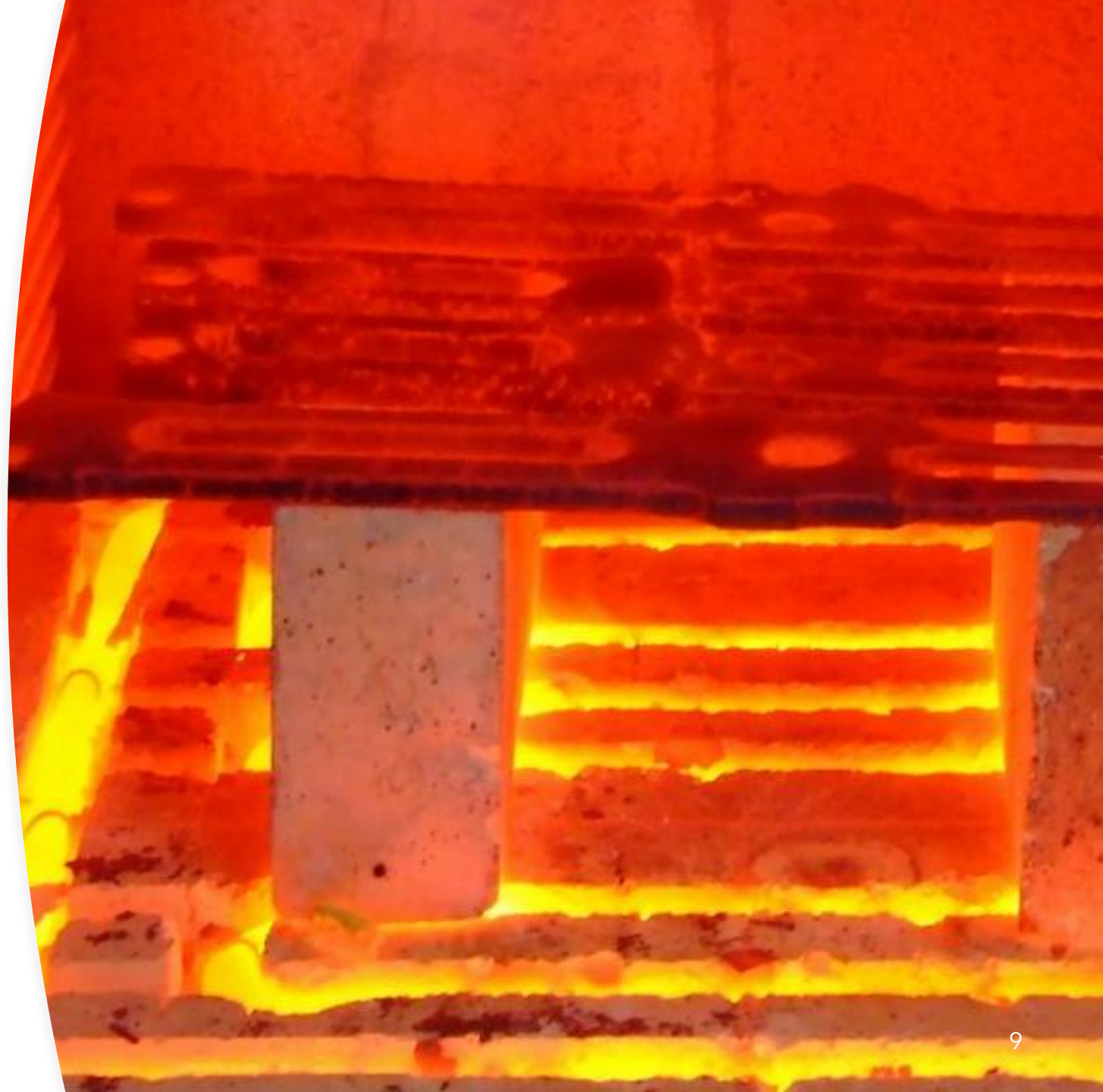
- Hill climber con una lista de nodos visitados previamente τ
1. Dado una asignación s , generar otra asignación n que no esté en la lista tabú
 2. Elegir el mejor vecino n que no está en la lista tabú
 1. $s \leftarrow n$
 3. $\tau = \tau \cup \{n\}$
- Regresar la mejor solución vista



Recocido simulado

- Escapar de óptimos locales permitiendo otro tipo de movimientos
 1. Iniciar en un estado s y t a un valor “alto”
 2. Seleccionar una asignación n del vecindario de s
 3. Si $f(n) > f(s)$
 1. $s \leftarrow n$
 4. De lo contrario
 1. $s \leftarrow n$ con probabilidad $e^{\left(\frac{f(n)-f(s)}{t}\right)}$
 5. Actualizar la temperatura $t \leftarrow \alpha t$

Regresar la mejor solución vista



Recocido simulado



Cuando la temperatura es alta:
búsqueda aleatoria



Cuando la temperatura es baja: es un
Hill climber



La temperatura baja
gradualmente $t_{k+1} = \alpha t_k$



Se puede demostrar su
optimalidad... con tiempo infinito

El algoritmo

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state

inputs: *problem*, a problem

schedule, a mapping from time to “temperature”

local variables: *current*, a node

next, a node

T, a “temperature” controlling prob. of downward steps

current $\leftarrow$ MAKE-NODE(INITIAL-STATE[*problem*])

for *t* $\leftarrow$ 1 **to** ∞ **do**

T $\leftarrow$ *schedule*[*t*]

if *T* = 0 **then return** *current*

next $\leftarrow$ a randomly selected successor of *current*

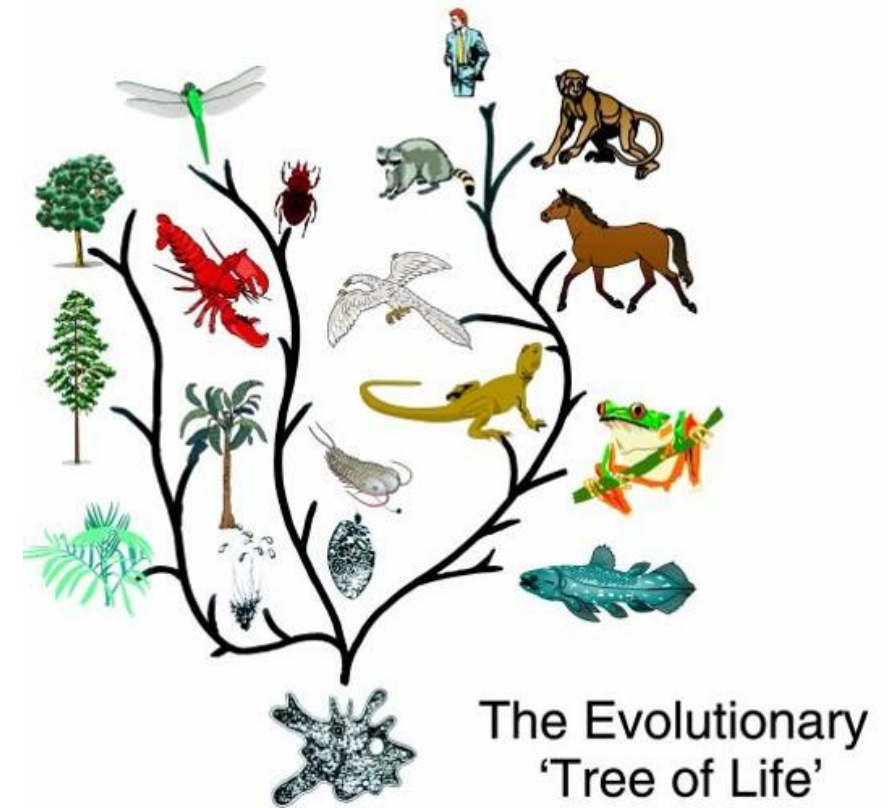
$\Delta E \leftarrow$ VALUE[*next*] – VALUE[*current*]

if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

else *current* $\leftarrow$ *next* only with probability $e^{\Delta E/T}$

Algoritmos evolutivos

- La evolución nos ha traído hasta aquí
- Se trata de un proceso natural bajo el cual aquellos individuos más aptos tienen mayor oportunidad de pasar sus genes a otras generaciones
- Supervivencia del más apto
- ¿Funcionará para resolver nuestros problemas?
- La idea: diseñar agentes que simulen el proceso evolutivo para resolver problemas





Elementos para simular el proceso evolutivo

Codificar las estructuras que se replicarán

Operaciones que afecten a los individuos

Una función de aptitud

Un mecanismo de selección



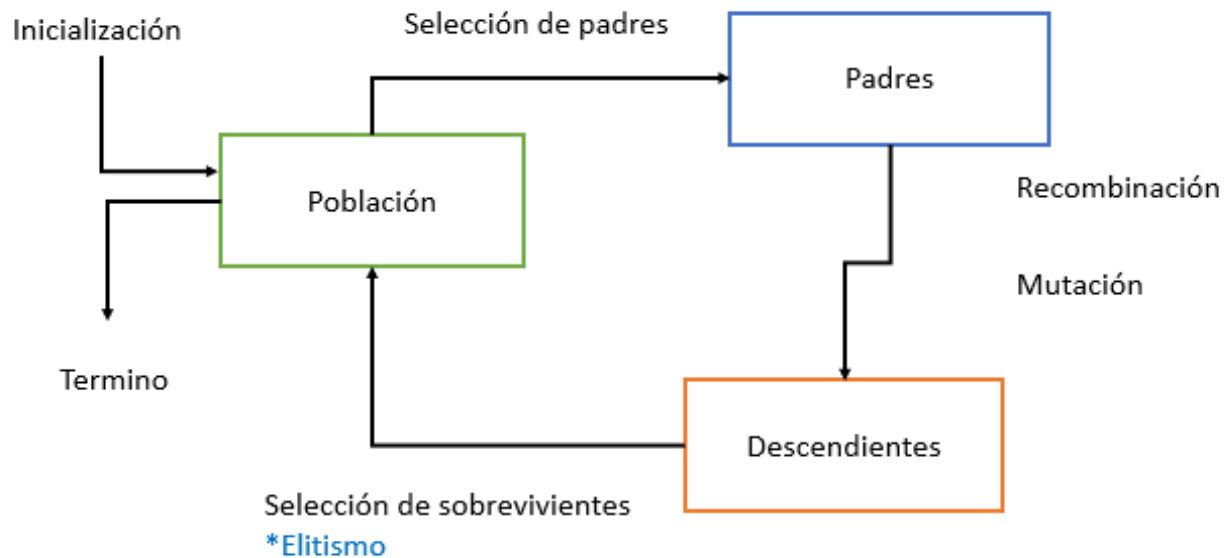
Tipos de algoritmos evolutivos

Estrategias evolutivas

Programación evolutiva

Algoritmos genéticos

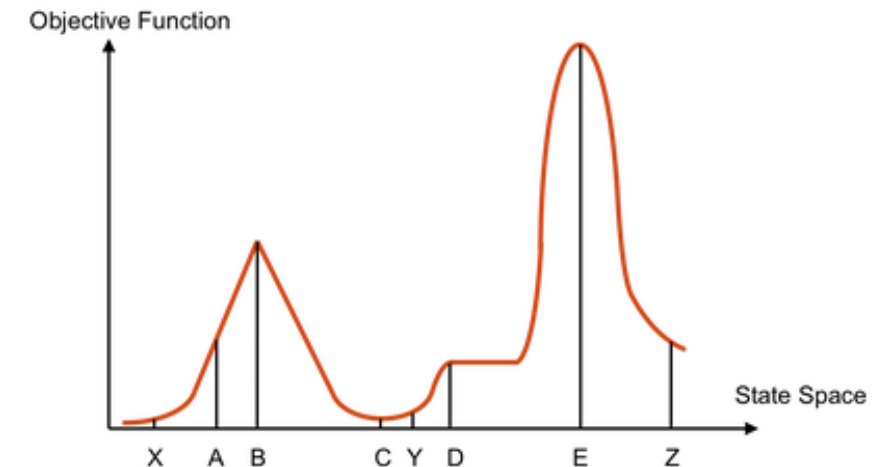
El algoritmo básico



- Generar (aleatoriamente) una población inicial.
- Calcular aptitud de cada individuo.
- Seleccionar (probabilísticamente) con base en aptitud.
- Aplicar operadores genéticos (cruza y mutación) para generar la siguiente población.
- Ciclar hasta que cierta condición se satisfaga

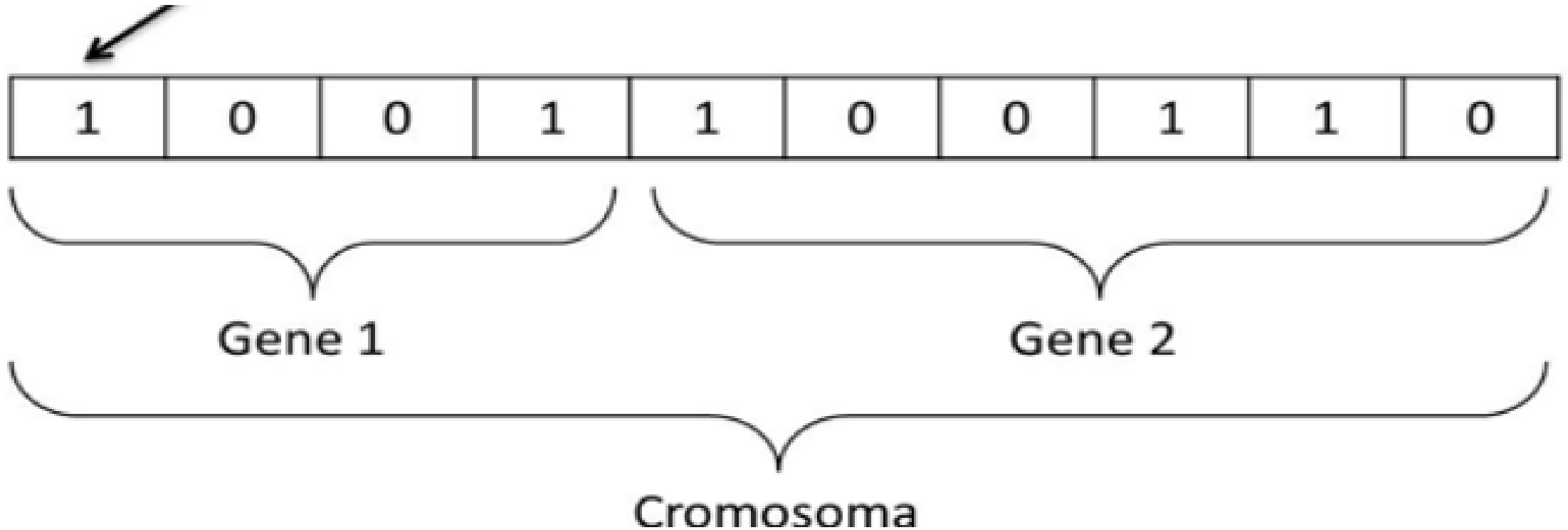
Selección por torneo

- Dado el tamaño del torneo s
- $padres = \emptyset$
- Hasta que se complete el número de padres μ
 - Elegir s individuos de forma aleatoria
 - Seleccionar al ganador i basado en su aptitud tal que $f(i) \leq f(j) \forall j \in \lambda$
 - $padres = padres \cup \{i\}$
- Regresar conjunto de padres



Representación

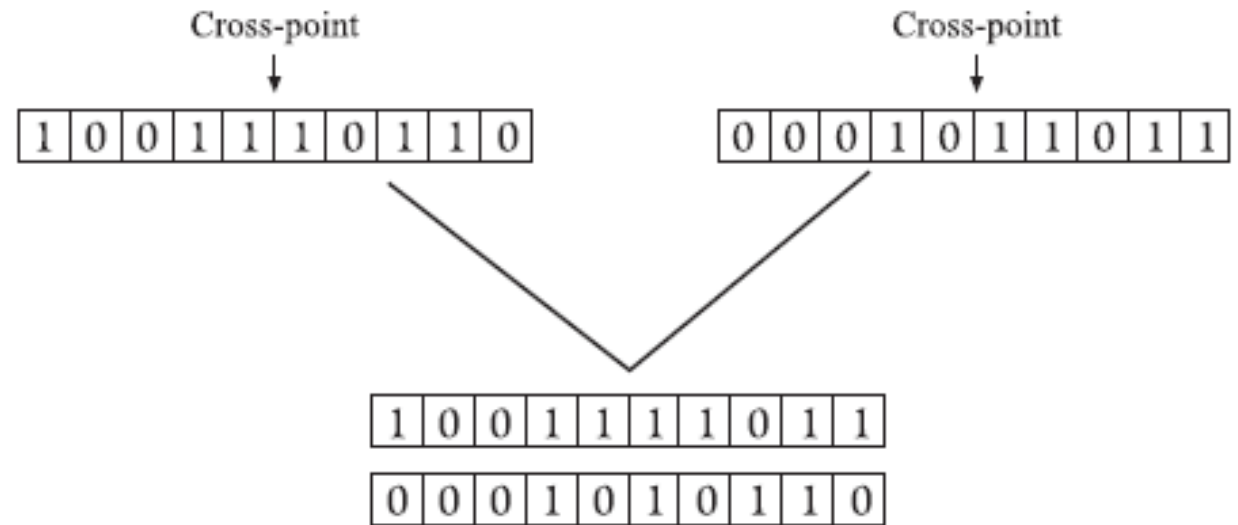
- La representación tradicional es la binaria
- A la cadena binaria se le llama "cromosoma"
- A cada cadena binaria que codifica una variable se le denomina "gene"
- Al valor dentro de esta posición se le llama "alelo"



Operadores: cruza de un punto

1. Se elige una posición en el cromosoma de forma aleatoria (cross-point)
2. El primer hijo tendrá los elementos hasta el cross-point del primer padre y del cross-point en adelante del segundo.
3. El segundo hijo tendrá los elementos hasta el cross-point del segundo padre y del cross-point en adelante del primero.

También existen cruza de dos puntos, tres, hasta de n puntos

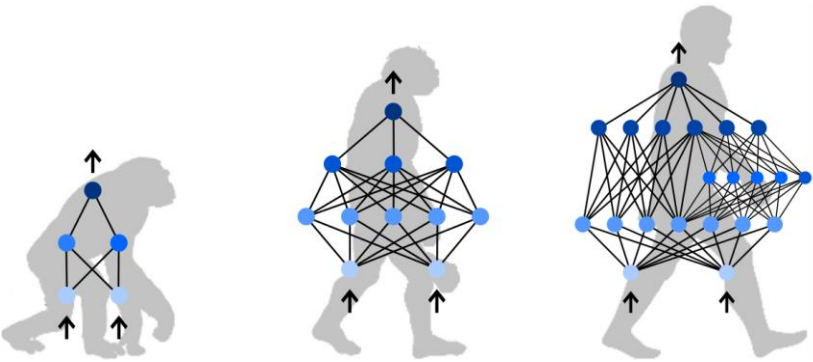


Operadores: mutación

- Para cada alelo del cromosoma, generar un número aleatorio y si es menor a P_m cambiar el valor de 0 a 1 y de 1 a 0 según corresponda
- Normalmente el valor de P_m es muy bajo, se ha sugerido $P_m = \frac{1}{L}$ donde L es el tamaño del cromosoma

Algunas notas

In practice, **genetic** algorithms have their place within the broad landscape of optimization methods (**Marler and Arora, 2004**), particularly for complex structured problems such as circuit layout or job-shop scheduling, and more recently for evolving the architecture of deep neural networks (**Miikkulainen *et al.*, 2019**). It is not clear how much of the appeal of **genetic** algorithms arises from their superiority on specific tasks, and how much from the appealing metaphor of evolution.

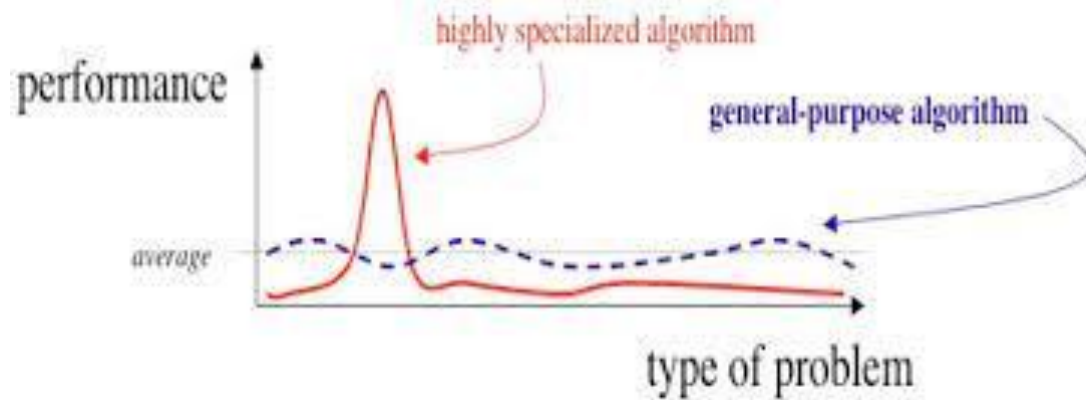


Algunas notas generales...

- Al tratar con problemas discretos o combinatorios se suele preferir búsqueda tabú y recocido simulado
- Para problemas continuos, los algoritmos evolutivos suelen dar buenos resultados, así como para problemas mixtos.
- Además... los evolutivos se han utilizado para:
 - generar software (programación genética)
 - aprendizaje maquina (aprendizaje máquina evolutivo)
 - Redes neuronales (neuroevolución)



No free lunch theorem



Para la otra vez...

- Búsqueda con adversarios



iimas

The End.